מחלקת תקשוב · רשתות · מדריך למעביר השיעור

הצפנה וזהות

הסבר מלא לכל שקופית: מה מראים, מה צריך להבין כדי להסביר את זה, אילו שאלות יעלו ואיפה חניכים נוטים לטעות.

30	כ-105 דקות	2 מפגשים
שקופיות	משך מומלץ	אפשר לפצל

לטכנאי שטח בסביבת Windows מנוהלת · חלק א׳ לשטח, חלק ב׳ רקע תיאורטי

מטרה, הכנה וחלוקת זמן

למי המדריך הזה

המדריך כתוב למי שמעביר את השיעור ולא בהכרח מגיע מאבטחת מידע. בכל שקופית יש הסבר רקע רחב יותר ממה שמופיע על המסך, כדי שתבינו מה אתם מציגים ותוכלו לענות כשחניך שואל ״למה״. לא צריך להקריא את הרקע לכיתה. קחו ממנו את מה שמתאים לקבוצה.

אחרי הרקע מופיעות, כשיש כאלה, שאלות שחניכים שואלים בדרך כלל, עם תשובה קצרה, וטעויות שכדאי לתפוס מוקדם.

מטרת השיעור

שהטכנאי יידע, מול כל כלי הצפנה שהוא פוגש בשטח, מה הכלי מוכיח ומה לא. בסוף השיעור חניך אמור להבדיל בין קידוד, הצפנה והסתרה; להסביר מה נותנים Hash, הצפנה סימטרית, הצפנה אסימטרית וחתימה; להצביע, מול שגיאת תעודה, על החוליה שנכשלה; ולהסביר למה PIN של ארבע ספרות מספיק בכרטיס חכם, אבל לא כסיסמה.

הכנה

- פתחו את המצגת ב-Chrome או ב-Edge במחשב שמחובר למקרן, ועברו על כל השקופיות. המצגת לא צריכה חיבור לרשת.
- הניווט: חצים או רווח, N פותח את הערות המדריך, F מסך מלא, A מכבה ומדליק אנימציות.
- כשהסמן בתוך תיבת טקסט, החצים והרווח כותבים בתיבה ולא מעבירים שקופית. Esc או לחיצה מחוץ לתיבה מחזירים את הניווט.
- מספרי השקופיות במדריך הם המספרים שבמונה בתחתית המסך (״30 / 2״). המספר הקטן שליד הכותרת בכל שקופית נמוך באחד, כי שקופית השער לא ממוספרת.
- אם יש בכיתה כרטיס חכם וקורא, הכינו תחנה שאפשר להריץ בה certutil -scinfo בשקופית 15. אם אין, מראים את הפקודה בלבד.

> **מה לא מלמדים בשיעור הזה**
>
> השיעור עוסק במה שכל כלי נותן ואיך הוא עובד. הוא לא מלמד הקמה או ניהול של רשות תעודות, ולא נוהל שחרור כרטיס חסום. מספר ניסיונות ה-PIN בהדגמה (שלושה) הוא להמחשה. המספר האמיתי ותהליך השחרור נקבעים במדיניות של מערכת הכרטיסים ביחידה, ואין להציג מספר אחר כעובדה.

חלוקת זמן מוצעת

חלק	שקופיות	מה מעבירים	דקות
פתיחה	1-3	קידוד, הצפנה והסתרה, סטגנוגרפיה	10
הכלים	4-8	Hash, סימטרית, אסימטרית, היברידית	15
תעודות	9-12	המנעול בדפדפן, חתימה, שרשרת, שגיאות	14
כרטיס חכם	13-15	אתגר וחתימה, PIN, איתור תקלות	9
תרגול וסיכום	16-17	אתב״ש וקיסר, כללים לשטח	10

חלק ב׳	18-20	פיצוח תדירויות, SHA-256 מבפנים	9
XOR ומפתח זרם	21-22	פנקס חד-פעמי, איך מפתח קצר מצפין הרבה	9
אלגוריתמים	23-26	AES, RSA, דיפי-הלמן, חתימה במספרים	14
סגירה	27-30	לחיצת היד של TLS 1.2, 1.3 מול 1.3, מגבלות, סיכום	15

חלק א׳ לוקח כ-58 דקות וחלק ב׳ כ-47. אם מפצלים, עוצרים אחרי שקופית 17 ופותחים את המפגש השני בשקופית 18. כשהזמן קצר, אפשר לוותר בחלק ב׳ על SHA-256 מבפנים (20) ועל החתימה במספרים (26). את XOR, מפתח הזרם, לחיצת היד ושקופית המגבלות (29) מעבירים בכל מקרה.

פתיחה: קידוד, הצפנה והסתרה

שקופית 1 · שער

השורה הצהובה בשער מתחלפת לאט בין ״הצפנה שומרת סוד״ לבין אותו משפט אחרי אתב״ש. אל תסבירו את זה עכשיו. נחזור לאתב״ש בתרגול שבשקופית 16, ואז אפשר לחזור לשער ולשאול אם מה שראו כאן היה הצפנה.

שקופית 2 · קידוד, הצפנה והסתרה

על המסך: תיבת טקסט ותיבת סיסמה, ומתחתן טבלה שמראה את אותו טקסט ארבע פעמים: בקידוד Hex, בקידוד Base64, מוצפן בסיסמה, ומוסתר בתמונה. העמודה השמאלית אומרת מי יכול לשחזר אותו.

רקע. בשלוש הפעולות יוצא משהו שאי אפשר לקרוא במבט, אבל רק בהצפנה צריך מפתח כדי לקרוא אותו בחזרה.

קידוד הוא דרך אחרת לכתוב את אותו מידע. הכללים שלו ציבוריים ואין בו שום סוד, ולכן כל אחד יכול להפוך אותו בחזרה. Hex כותב כל בית כשתי ספרות הקסדצימליות. Base64 לוקח כל שלושה בתים והופך אותם לארבעה תווים מתוך אלפבית של 64 (אותיות לטיניות גדולות וקטנות, ספרות, ״+״ ו-״/״), כך שהתוצאה ארוכה בערך בשליש מהמקור, וכשמספר הבתים לא מתחלק בשלוש מוסיפים בסוף ״=״ או ״==״. כל אחד מהם קיים כדי להעביר בתים בערוץ שתומך רק בטקסט: מצורפים בדוא״ל, קובצי JSON, תעודות בפורמט PEM. בשורת ה-Hex רואים שכל אות עברית תופסת שני בתים ב-UTF-8 (למשל d7 a0 לנ׳), וספרה או רווח תופסים בית אחד.

הצפנה משתמשת במפתח. מי שאין לו את המפתח לא יכול להחזיר את המידע, גם אם הוא יודע בדיוק באיזו שיטה השתמשו. בהדגמה המפתח נגזר מהסיסמה: הסיסמה עוברת פונקציה איטית בכוונה (PBKDF2, עם ״מלח״ אקראי), והתוצאה משמשת מפתח ל-AES. בהדגמה יש אלף סבבים כדי שהיא תגיב מיד. במערכות אמיתיות משתמשים במאות אלפים. ההצפנה עצמה היא AES במצב מונה, אותה שיטה שמוסברת בשקופית 22. בחלק א׳ לא צריך להיכנס לזה. מספיק להראות שבלי הסיסמה הנכונה יוצא רעש. ברירת המחדל בשדה ״נסו לפענח״ היא הסיסמה עם אות ראשונה קטנה, ולכן בהתחלה מופיע ✗ ורעש. תקנו את האות ויופיע ✓ והטקסט. כל טעינה של המצגת מגרילה מלח ומספר חד-פעמי חדשים, ולכן הטקסט המוצפן שונה בכל פעם, גם לאותו טקסט ולאותה סיסמה.

הסתרה (סטגנוגרפיה) מחביאה את המידע כמו שהוא, בלי לשנות אותו. התמונה בטבלה נראית רגילה, והטקסט כתוב בביט הנמוך של כל ערך צבע. אין בזה מפתח, וההגנה היחידה היא שהצד השני לא חושד שיש מסר. מי שיודע שיש מסר ואיפה לחפש קורא אותו מיד, כמו בשורה שליד התמונה. בשקופית הבאה רואים את זה מקרוב.

בשטח. פוגשים Base64 בקובצי הגדרות ובסקריפטים שבהם מישהו ״הסתיר״ סיסמה. גם הכותרת Authorization: Basic בפרוטוקול HTTP היא שם משתמש וסיסמה ב-Base64, ולכן בלי TLS כל מי שמאזין לתעבורה קורא אותם.

שאלה	תשובה
אם Base64 לא מגן על כלום, למה משתמשים בו?	כדי להעביר בתים בערוץ שמקבל רק טקסט, למשל בדוא״ל או ב-JSON.
איך מזהים Base64?	רק אותיות לטיניות, ספרות, ״+״ ו-״/״, אורך שמתחלק בארבע, ולפעמים ״=״ בסוף. הכי פשוט לנסות לפענח אותו.
אז ההסתרה בטוחה יותר מהצפנה?	לא. היא רק מסתירה שיש מסר. אפשר לשלב ולהצפין לפני שמסתירים, וכך גם מי שמוצא את המסר רואה רעש.

טעות נפוצה

״זה נראה כמו ג׳יבריש, אז זה מוצפן״. שורות ה-Hex וה-Base64 נראות אקראיות בדיוק כמו השורה המוצפנת. השאלה היא אם צריך מפתח כדי לקרוא את זה.

שקופית 3 · סטגנוגרפיה

על המסך: שתי תמונות שנראות זהות, באחת מהן מוסתר מסר. הכפתור ״הגבר ביט אחרון״ מראה איפה.

רקע. כל פיקסל בתמונה צבעונית מורכב משלושה ערכים, אדום, ירוק וכחול, וכל ערך הוא מספר בין 0 ל-255 (בית אחד, שמונה ביטים). הביט האחרון של כל ערך משנה את הצבע ביחידה אחת לכל היותר, וזה שינוי שהעין לא מבחינה בו. אם כותבים את ביטי המסר במקום הביטים האחרונים, התמונה נראית בדיוק אותו דבר, ובכל פיקסל אפשר להחביא שלושה ביטים של מסר.

״הגבר ביט אחרון״ מציג רק את הביט האחרון של כל ערך ומגדיל אותו פי 255, כך ש-0 נשאר שחור ו-1 הופך לבהיר. בתמונה המקורית, שעשויה ממשטחים אחידים, הביט האחרון קבוע בכל משטח. בתמונה עם המסר מופיע בראש התמונה פס רעש, בדיוק בפיקסלים שבהם נכתב המסר. שנו את הטקסט בתיבה וראו שהפס מתארך.

ההבדל בין סטגנוגרפיה להצפנה: הצפנה לא מסתירה שיש תקשורת. מי שמאזין רואה שעובר משהו מוצפן, רק לא מבין מה. סטגנוגרפיה מסתירה את עצם קיום המסר, אבל לא מגינה על התוכן: מי שיודע לחפש קורא הכול. אפשר לשלב ולהצפין את המסר לפני שמסתירים אותו.

בשטח. לטכנאי זה בעיקר איום. מידע יכול לצאת מהרשת בתוך תמונות או קובצי קול שנראים תמימים, וכלי סינון שבודק רק את סוג הקובץ לא יראה את זה.

שאלה	תשובה
אם שולחים את התמונה בוואטסאפ, המסר נשאר?	בדרך כלל לא. אפליקציות דוחסות תמונות מחדש (למשל ל-JPEG), והדחיסה משנה בדיוק את הביטים האחרונים.
אפשר לגלות שיש מסר מוסתר?	לפעמים, בניתוח סטטיסטי של התמונה, בעיקר כשהתמונה פשוטה כמו בהדגמה. בתמונה מצולמת רגילה זה קשה הרבה יותר.

Hash, הצפנה סימטרית, אסימטרית והיברידית

שקופית 4 · הכלים המודרניים

על המסך: טבלה של ארבעה כלים, מה כל אחד נותן ומה המגבלה שלו. כל שורה מקבלת בהמשך שקופית משלה.

רקע. כותרת המשנה, ״השיטות ציבוריות ונבדקו שנים, והסוד היחיד הוא המפתח״, היא עקרון קרקהוף, מהמאה ה-19. האלגוריתמים שבשימוש היום (SHA-256, AES, RSA) מתוארים בתקנים פתוחים, וחוקרים בכל העולם ניסו לשבור אותם במשך שנים. אלגוריתם סודי לא עבר בדיקה כזו, וכשהוא דולף, וזה קורה, צריך להחליף את כל המערכת. אם רק המפתח סודי, מספיק להחליף את המפתח.

הקופסה בתחתית השקופית מגדירה את שלוש התכונות שכדאי לבדוק מול כל כלי: **סודיות** (מי שאין לו מפתח לא קורא את התוכן), **שלמות** (אם מישהו שינה את המידע בדרך, זה מתגלה) ו**אימות** (אפשר להוכיח מי שלח או עם מי מדברים). Hash נותן שלמות בלבד, הצפנה סימטרית נותנת סודיות, וחתימה נותנת שלמות ואימות אבל לא סודיות. בפועל משלבים כמה כלים יחד.

עברו על הטבלה שורה-שורה: מה כל כלי נותן ומה לא.

- **Hash** מייצר טביעת אצבע באורך קבוע מכל קלט. אין בו מפתח ואין דרך חזרה. הוא מגלה שינוי, אבל לא מסתיר כלום ולא מוכיח מי יצר את המידע.
- **הצפנה סימטרית** משתמשת באותו מפתח להצפנה ולפענוח. מהירה ומתאימה לקבצים גדולים; הבעיה היא איך להעביר את המפתח לצד השני.
- **הצפנה אסימטרית** משתמשת בזוג מפתחות: ציבורי שאפשר לפרסם, ופרטי שנשאר אצל הבעלים. היא פותרת את בעיית העברת המפתח, אבל איטית מכדי להצפין בה נתונים רבים.
- **חתימה** מבוססת על אותם זוגות מפתחות, בכיוון ההפוך. היא מוכיחה מי חתם ושהתוכן לא השתנה, ולא מסתירה אותו.

שאלות לבדיקת הבנה, מהערות השקופית: מה מגן על סיסמאות שמורות בשרת (Hash), מה מצפין קובץ גדול (סימטרית), איך מעבירים את המפתח הסימטרי (אסימטרית), ואיך יודעים שעדכון תוכנה באמת הגיע מהיצרן (חתימה).

שאלה	תשובה
למה לא מצפינים סיסמאות בשרת במקום לשמור Hash?	כי מי שמשיג את מפתח ההצפנה מפענח את כל הסיסמאות. עם Hash השרת לא צריך את הסיסמה עצמה בשום שלב: הוא מחשב Hash למה שהוקלד ומשווה.

טעות נפוצה

לקרוא ל-Hash ״הצפנה חד-כיוונית״ ולחשוב שהוא מסתיר את הקלט. Hash של סיסמה קצרה או נפוצה נשבר בניחוש: מחשבים Hash למיליוני סיסמאות ומשווים.

שקופית 5 · Hash בשטח

על המסך: שני משפטים שנבדלים בספרה אחת (״500 ליטר״ מול ״900 ליטר״), ה-Hash של כל אחד, ומונה של הביטים השונים.

רקע. SHA-256 מחזיר תמיד 256 ביטים, שנכתבים כ-64 תווים הקסדצימליים, בין אם הקלט הוא מילה אחת ובין אם הוא קובץ של כמה ג׳יגה-בייט. אותו קלט נותן תמיד אותו פלט, אבל שינוי של תו אחד הופך בערך מחצית מהביטים. זה נקרא

אפקט המפולת. בהדגמה המונה מראה מספר סביב 128, והתווים השונים מסומנים בצהוב כמעט לאורך כל השורה. המשמעות: אי אפשר ״לכוון״ Hash בשינויים קטנים, ואי אפשר ללמוד מה-Hash משהו על הקלט.

בדיקת קובץ שהורד. מחשבים Hash לקובץ ומשווים לערך שהיצרן פרסם. ב-Windows יש לזה שני כלים מובנים: Get-FileHash ב-PowerShell (ברירת המחדל שלו היא SHA-256) ו-certutil -hashfile. אם הערכים שונים, הקובץ השתנה או נפגם בהורדה. הבדיקה שווה משהו רק אם את הערך המפורסם לקחתם ממקור שאתם סומכים עליו. מי שהחליף את הקובץ באתר יכול להחליף גם את ה-Hash שלידו.

שמירת סיסמאות. שרת תקין לא שומר את הסיסמה. הוא שומר Hash שלה, יחד עם ״מלח״: ערך אקראי שונה לכל משתמש שמצרפים לסיסמה לפני החישוב. בזכות המלח לשני משתמשים עם אותה סיסמה יש Hash שונה, וטבלאות מוכנות מראש של סיסמאות נפוצות לא עוזרות. שרתים טובים גם משתמשים בפונקציה שאיטית בכוונה (PBKDF2, bcrypt, Argon2), כדי שכל ניחוש יעלה זמן. זה נקרא key stretching, ״מתיחת מפתח״. אל תבלבלו את זה עם מפתח הזרם שבשקופית 22: כאן מאטים את הניחוש של סיסמה, ושם מייצרים רצף ארוך ממפתח קצר.

MD5 ו-SHA-1. בשניהם נמצאו התנגשויות, כלומר שני קבצים שונים עם אותו Hash: ב-MD5 כבר באמצע שנות ה-2000, וב-SHA-1 בהדגמה ציבורית ב-2017. לבדיקה אם קובץ נפגם בהעתקה הם עדיין עובדים. לאבטחה, כלומר כשמישהו עלול לזייף בכוונה, משתמשים ב-SHA-256 ומעלה.

שאלה	תשובה
אם Hash חד-כיווני, איך פורצים סיסמאות?	לא הופכים את ה-Hash, מנחשים: מחשבים Hash לכל סיסמה אפשרית ומשווים. לכן סיסמה ארוכה ואקראית חשובה, וזה גם הקשר לשקופית 14.
יכולים להיות שני קבצים עם אותו SHA-256?	מתמטית כן, כי יש אינסוף קלטים ומספר סופי של פלטים. בפועל אף אחד לא מצא זוג כזה, ואין דרך ידועה לחפש אחד.

שקופית 6 · הצפנה סימטרית ובעיית הפצת המפתח

על המסך: רשת של משתתפים, וסליידר שמראה כמה מפתחות צריך כשכל זוג מדבר בסוד.

רקע. בהצפנה סימטרית אותו מפתח מצפין ומפענח. האלגוריתם הנפוץ היום הוא AES, עם מפתח של 128 או 256 ביט. הוא מהיר מאוד, ובמעבדים מודרניים יש לו אפילו פקודות ייעודיות בחומרה, ולכן מצפינים בו דיסקים שלמים (BitLocker משתמש ב-AES) וזרמי וידאו בלי שמרגישים.

הבעיה היא המפתח. כל זוג שרוצה לדבר בסוד צריך מפתח משלו, וכל מפתח צריך להגיע לשני הצדדים בלי שאף אחד יירט אותו. עם N משתתפים צריך N(N-1)/2 מפתחות. הזיזו את הסליידר לאט: 6 משתתפים צריכים 15 מפתחות, 40 צריכים 780, ו-300 כבר צריכים 44,850. את הבעיה הזו פותרת ההצפנה האסימטרית בשקופית הבאה.

בשטח. BitLocker, ארכיון מוצפן עם סיסמה ורשת Wi-Fi עם סיסמה משותפת הם כולם הצפנה סימטרית. בכולם, מי שמחזיק את הסוד יכול גם לקרוא וגם לכתוב, ואין דרך לדעת מי מבין המחזיקים שלח מה. ב-Wi-Fi עם סיסמה משותפת (WPA2-Personal) מי שמכיר את הסיסמה וקלט את רגע החיבור של מכשיר אחר יכול לחשב את המפתח של אותו מכשיר ולפענח את התעבורה שלו. זה מה שמתכוונים אליו ב״בתנאים מסוימים״ שבהערות. WPA3 סוגר את הפרצה הזו.

שאלה	תשובה
למה לא לתת לכולם אותו מפתח?	כי דליפה אחת חושפת את כולם, וכל מחזיק יכול להתחזות לכל אחד אחר.

שקופית 7 · הצפנה אסימטרית

על המסך: אנימציה של מנעול פתוח שעובר מדנה ליוסי, נסגר על קופסה וחוזר לדנה, שרק היא יכולה לפתוח.

רקע. בהצפנה אסימטרית כל משתתף מייצר זוג מפתחות שקשורים זה לזה מתמטית. מה שהמפתח הציבורי נועל, רק המפתח הפרטי פותח. את הציבורי אפשר לפרסם לכולם, כמו מנעול פתוח שתולים על לוח המודעות. מי שמעתיק אותו

יכול רק לסגור בו. גם אי אפשר לחשב את המפתח הפרטי מהמפתח הציבורי, לפחות לא בזמן סביר. האלגוריתמים הנפוצים הם RSA ו-ECC (הצפנה על עקומים אליפטיים, עם מפתחות קצרים יותר לאותה רמת ביטחון).

לכן צריך הרבה פחות מפתחות: עם N משתתפים מספיקים N זוגות מפתחות, ואף מפתח פרטי לא צריך לעבור בשום ערוץ. החיסרון הוא מהירות: פעולות אסימטריות איטיות בסדרי גודל מסימטריות, ולכן מצפינים בהן רק דברים קטנים: מפתח סימטרי או Hash.

בתחתית השקופית מופיעה חתימה, בכיוון ההפוך: מה שנחתם במפתח הפרטי, כל אחד בודק במפתח הציבורי. נחזור לזה בשקופית 10.

שאלה	תשובה
אם המפתח הציבורי גלוי לכולם, מה מונע ממישהו לפרסם מפתח משלו ולטעון שהוא דנה?	בשביל זה יש תעודות: מסמך חתום שקושר מפתח ציבורי לשם. זה הנושא של שקופיות 9-12.

שקופית 8 · בפועל משלבים את שתי השיטות

על המסך: לקוח ושרת. למעלה הסכמה חד-פעמית על מפתח, ולמטה זרם המידע בהצפנה סימטרית. בתחתית פסים שמראים סדר גודל של מהירות.

רקע. כמעט כל פרוטוקול מאובטח עובד כך: פעם אחת, בתחילת החיבור, משתמשים בשיטה אסימטרית כדי ששני הצדדים יגיעו לאותו מפתח סימטרי, ומשם כל המידע מוצפן בסימטרית. בדוא"ל מוצפן זה כמעט מילולי: מצפינים את ההודעה במפתח סימטרי אקראי, ואת המפתח הזה מצפינים במפתח הציבורי של הנמען. ב-TLS 1.3 ההסכמה נעשית בדיפי-הלמן (שקופית 25): שני הצדדים מחשבים את אותו סוד בלי שאף אחד מהם שולח אותו.

> **טעות נפוצה**
>
> לתת מספר מדויק ליחס המהירויות ("פי אלף"). אין מספר כזה: הוא תלוי באלגוריתם, בגודל המפתח ובחומרה. אומרים "סדרי גודל", וזה מספיק כדי להבין למה משלבים.

המנעול בדפדפן, חתימה ותעודות

שקופית 9 · מה המנעול בדפדפן מבטיח

על המסך: דפדפן ושרת עם מנהרת TLS ביניהם, ומאזין באמצע. בצד: מה המאזין עדיין רואה ומה לא. מתחת: מה המנעול מבטיח (ירוק), מה הוא לא מבטיח (אדום), וחריג ארגוני (כתום).

רקע. TLS הוא הפרוטוקול שמאחורי HTTPS. המנעול בשורת הכתובת אומר שני דברים: שהתעבורה בין הדפדפן לשרת מוצפנת ומוגנת משינוי, ושהשרת הוכיח שהוא מחזיק את המפתח הפרטי של תעודה תקפה לשם שמופיע בשורת הכתובת. זה הכול.

המנעול לא אומר שהאתר הגון. גם לאתר מתחזה יכולה להיות תעודה תקפה, לשם שלו. אם המשתמש נכנס ל-portal-unit.com במקום ל-portal.unit, הוא יקבל מנעול תקין לאתר הלא נכון.

גם כשהתעבורה מוצפנת, המאזין רואה לא מעט: כתובות IP, פורט (443), את שם האתר שנשלח בתחילת החיבור (SNI, שקופית 28), וכמה מידע עובר ומתי. הוא לא רואה את כתובת הדף המלאה, את התוכן, טפסים, סיסמאות ועוגיות.

החריג הארגוני. בארגונים רבים יש "בדיקת תעבורה מוצפנת": רכיב ברשת פותח את ה-TLS, בודק את התוכן ומצפין מחדש בתעודה שהוא מייצר בעצמו. כדי שהדפדפן לא יתריע, התחנות מקבלות מראש תעודת שורש פנימית (בדרך כלל במדיניות דומיין). התוצאה: מנעול תקין, למרות שיש מישהו באמצע. זה חוקי ומכוון, ולכן ברשת ארגונית המנעול לא מוכיח שאין בדיקה בדרך. רואים את זה כשפותחים את התעודה: המנפיק הוא רשות פנימית ולא רשות ציבורית מוכרת.

שאלה	תשובה
אז אם יש HTTPS, למה עדיין יש פישינג?	כי התעודה מוכיחה רק את השם. אתר מתחזה מקבל תעודה לשם שלו, והמשתמש לא שם לב שהשם שונה.
הארגון רואה את מה שאני מקליד באתרים?	רכיב הבדיקה רואה את התוכן הפתוח של התעבורה שהוא בודק. מה נבדק ומה לא נקבע במדיניות הארגון.

שקופית 10 · חתימה דיגיטלית

על המסך: מסמך ("מאושרת הצבת אנטנה בעמדה 7..."), כפתור לחתימה וכפתור לבדיקה, ושרשרת: מסמך, Hash, נעילה במפתח הפרטי, חתימה.

רקע. כדי לחתום, מחשבים Hash למסמך ונועלים את ה-Hash במפתח הפרטי. התוצאה היא החתימה, והיא נשלחת יחד עם המסמך. הבודק פותח את החתימה במפתח הציבורי ומקבל את ה-Hash המקורי, ואז מחשב Hash למסמך שקיבל. אם השניים שווים, המסמך לא השתנה, ורק מי שמחזיק את המפתח הפרטי יכול היה לייצר את החתימה.

ההדגמה. חתמו, ואז שנו את 7 ל-8 ובדקו: אדום. החזירו את 7: ירוק שוב. הוסיפו רווח בסוף: אדום, כי גם שינוי שהעין לא רואה משנה את ה-Hash. ההדגמה משתמשת ב-SHA-256 אמיתי ובמפתח RSA זעיר כדי שהמספרים יהיו קריאים. את החשבון רואים בשקופית 26.

חתימה לא מסתירה את המסמך. כל אחד יכול לקרוא אותו. מה שהיא מוכיחה זה מקור ושלמות.

התיאור של "נעילת" ה-Hash ו"פתיחתו" מדויק ל-RSA. בחתימות על עקומים אליפטיים (ECDSA), שנפוצות גם הן בתעודות, החשבון אחר ואי אפשר "לפתוח" את החתימה, אבל התוצאה זהה: רק בעל המפתח הפרטי יכול לחתום, וכל אחד יכול לבדוק.

בשטח. עדכוני Windows, תוכנות ומנהלי התקן חתומים. Windows מציג את שם החותם בחלון האישור, מזהיר על "מפרסם לא ידוע" כשאין חתימה, ובגרסאות 64 ביט מסרב לטעון מנהל התקן לא חתום. קובץ חתום שנפתח, שונה

ונשמר מחדש מאבד את החתימה.

שאלה	תשובה
זה כמו חתימה סרוקה?	לא. חתימה סרוקה היא תמונה שאפשר להעתיק לכל מסמך. חתימה דיגיטלית נגזרת מהתוכן עצמו ולא מתאימה לשום מסמך אחר.
מה אם מישהו גונב את המפתח הפרטי?	הוא יכול לחתום בשם הבעלים. לכן מבטלים תעודות שמפתח שלהן דלף, ולכן שומרים מפתחות פרטיים בכרטיס חכם (שקופית 13).

שקופית 11 · שרשרת תעודות

על המסך: שלוש חוליות (שורש, ביניים, שרת), שעון התחנה, ושישה תרחישים שאפשר להריץ ולראות איפה הבדיקה נעצרת.

רקע. תעודה היא מסמך שקושר מפתח ציבורי לשם, וחתום בידי מי שהנפיק אותה. בתעודה יש בין השאר את השמות שהיא תקפה להם (ברשימה שנקראת Subject Alternative Name), תאריכי תוקף, המפתח הציבורי וחתימת המנפיק.

הדפדפן לא מכיר את תעודת השרת מראש. הוא בודק שהיא חתומה בידי רשות ביניים, שרשות הביניים חתומה בידי שורש, ושהשורש נמצא במאגר האמון של Windows. השורש חתום בידי עצמו, וסומכים עליו רק כי הוא מותקן במאגר. בדרך הדפדפן בודק גם שהשם תואם, שכל תעודה בתוקף לפי שעון התחנה, ולפעמים גם שהתעודה לא בוטלה. הבדיקה נעצרת בחוליה הראשונה שנכשלת.

ההדגמה. הריצו קודם ״תקין״ כדי לראות את סדר הבדיקות, ואז את שאר התרחישים, ולפני כל אחד בקשו ניחוש איזו חוליה תיכשל. שני תרחישים שכדאי להתעכב עליהם:

- **שעון שגוי בתחנה** נותן בדיוק את אותה שגיאה כמו תעודה שפג תוקפה, כי הדפדפן משווה את תאריכי התעודה לשעון המקומי. התיקון הוא בתחנה, לא בשרת.
- **חסרה תעודת ביניים** קורה כשהשרת שולח רק את התעודה שלו. חלק מהלקוחות יודעים להוריד את תעודת הביניים בעצמם, לפי כתובת שמופיעה בתעודה, וחלק לא. לכן זו תקלה ש״עובדת אצלי״ ונכשלת אצל אחר.

בשטח. תחנות שמחוברות לדומיין מקבלות את השורש הפנימי של הארגון דרך מדיניות. תחנה שלא בדומיין, או תחנה שהמדיניות לא הגיעה אליה, תקבל שגיאת ״שורש לא מוכר״ בכל אתר פנימי.

שאלה	תשובה
אם השורש חתום בידי עצמו, מה מונע ממני לייצר שורש משלי?	כלום, אבל הוא לא יהיה במאגר האמון של אף אחד, ולכן אף דפדפן לא יסמוך עליו.
מה קורה אם רשות תעודות נפרצת?	התוקף יכול להנפיק תעודה ״תקינה״ לכל שם. בשביל זה יש ביטול תעודות, ומאגר אמון שמנוהל ומתעדכן.

שקופית 12 · שגיאות תעודה נפוצות ואיפה בודקים

על המסך: טבלה של ארבע שגיאות, הסיבה האופיינית לכל אחת, ואיפה בודקים.

רקע. כל שורה בטבלה מתאימה לתרחיש מבודק השרשרת. קודי השגיאה הם של Chrome ו-Edge, ובדפדפנים אחרים הנוסח שונה.

- **ERR_CERT_DATE_INVALID**: התעודה פגה, או שהשעון בתחנה שגוי. פותחים את התעודה (מהסמל שבתחילת שורת הכתובת, מנעול ב-Edge או סמל הגדרות ב-Chrome, או מהודעת השגיאה) ובודקים את התאריכים, ובתחנה מריצים w32tm /query /status לבדיקת מקור הזמן.

- **ERR_CERT_COMMON_NAME_INVALID**: השם בשורת הכתובת לא מופיע ברשימת השמות של התעודה. כמעט תמיד הסיבה בצד המשתמש: נכנס בכתובת IP או בשם הקצר (portal) במקום בשם המלא שבתעודה (portal.unit.local). בודקים ב-Details את Subject Alternative Name. דפדפנים מודרניים מסתכלים רק על הרשימה הזו ולא על שדה השם הישן (CN).
- **ERR_CERT_AUTHORITY_INVALID**: הדפדפן לא הצליח להגיע לשורש שהוא סומך עליו. או שהשורש הפנימי לא מותקן בתחנה, או שהשרת לא שולח את תעודת הביניים. בודקים בחלון התעודה את השרשרת (בחלון של Windows זו הלשונית Certification Path), ובמאגר התעודות של המחשב: certlm.msc, תחת Trusted Root Certification Authorities.
- **"לא ניתן לבדוק ביטול"**: בתעודה יש כתובת של רשימת ביטולים (CRL) או של שרת OCSP, והתחנה לא מגיעה אליה, למשל ברשת סגורה. ההודעה הזו נפוצה יותר בתוכנות שמשתמשות במנגנון התעודות של Windows (לקוחות VPN, Outlook) מאשר בדפדפנים. certutil -verify -urlfetch server.cer בודק את כל השרשרת ואת זמינות כתובות הביטול, ומראה איפה נכשל.

> **לא לוחצים "המשך בכל זאת"**
>
> זה בדיוק המצב שהתעודה נועדה לעצור. מתעדים את השגיאה, את השם שבשורת הכתובת ואת התחנה, ומתקנים את החוליה שנכשלה.

שאלה	תשובה
למה השגיאה רק בתחנה אחת?	שעון שגוי, שורש שחסר במאגר של התחנה הזו, או מטמון ישן. בודקים w32tm ו-certlm.msc בתחנה התקולה ומשווים לתחנה תקינה.

כרטיס חכם, PIN ואיתור תקלות

שקופית 13 · כרטיס חכם

על המסך: לולאה בין בקר הדומיין, התחנה והכרטיס: אתגר אקראי, חתימה בתוך השבב, והחתימה חוזרת לבדיקה.

רקע. בכרטיס יש שבב עם מפתח פרטי שנוצר בתוכו ולא ניתן לייצוא, ותעודה שקושרת את המפתח הציבורי המתאים לשם המשתמש. כשמתחברים, השרת לא מקבל סיסמה. הכרטיס מוכיח שהוא מחזיק את המפתח הפרטי בכך שהוא חותם, בתוך השבב, על נתונים שמשתנים בכל כניסה. בקר הדומיין בודק את החתימה במפתח הציבורי שבתעודה, אחרי שבדק שהתעודה עצמה תקינה ושייכת לשרשרת שהוא סומך עליה.

לכניסה צריך גם את הכרטיס (משהו שיש לי) וגם את ה-PIN (משהו שאני יודע). אחד מהם לבד לא מספיק כדי להיכנס.

ב-Windows. כניסה לדומיין בכרטיס חכם עוברת ב-Kerberos, בגרסה מבוססת תעודות שנקראת PKINIT. התהליך מורכב יותר מהתרשים: התחנה חותמת על נתונים שכוללים את השעה ומספר חד-פעמי, ובקר הדומיין מחזיר גם הוא תעודה שהתחנה בודקת. בשטח זה אומר ששרשרת התעודות צריכה להיות תקינה בשני הכיוונים, ושהשעון חשוב: Kerberos דוחה בקשות כשההפרש בין התחנה לבקר הדומיין גדול מחמש דקות (ברירת המחדל).

הקלטה של הכניסה לא עוזרת לתוקף, כי הנתונים שעליהם חותמים שונים בכל פעם, וחתימה ישנה לא תתקבל.

שקופית 14 · למה PIN של 4 ספרות מספיק בכרטיס חכם

על המסך: מקלדת PIN לכרטיס מדומה, עם מונה ניסיונות. ה-PIN לתרגול הוא 2580.

רקע. ה-PIN נבדק בתוך הכרטיס, והכרטיס עצמו סופר ניסיונות שגויים. אחרי מספר קבוע של טעויות הוא ננעל, ואי אפשר לעקוף את המונה מבחוץ. לכן מספר הניחושים האפשריים מוגבל: עם שלושה ניסיונות מתוך 10,000 צירופים, הסיכוי לנחש הוא שלושה מתוך עשרת אלפים, כלומר 0.03%.

בסיסמה המצב שונה לגמרי. מי שגנב קובץ של Hash-ים של סיסמאות מנחש במחשב שלו, בלי מונה ובלי נעילה. בסוגי Hash מהירים, כמו זה ש-Windows משתמש בו לשמירת סיסמאות מקומיות, מעבד גרפי אחד בודק מיליארדי ניחושים בשנייה. שם ההגנה היחידה היא סיסמה ארוכה ואקראית.

ה-PIN לא יוצא מהתחנה ולא מגיע לשרת. הוא משמש רק לפתיחת הכרטיס. אותו עיקרון נמצא ב-Windows Hello: ה-PIN פותח מפתח ששמור בשבב האבטחה של המחשב (TPM), והשבב מגביל את מספר הניסיונות.

ההדגמה. הזינו שלושה PIN שגויים (אפשר לתת לחניך לנחש). אחרי השלישי הכרטיס ננעל, וגם ה-PIN הנכון כבר לא פותח. ״כרטיס חדש״ מאפס את ההדגמה.

> **מה לא אומרים**
>
> מספר הניסיונות בהדגמה הוא להמחשה. בכרטיסים אמיתיים הוא נקבע במדיניות, ובדרך כלל נע בין 3 ל-10. כרטיס חסום משתחרר רק בתהליך מנהלתי, לפי נוהל היחידה. הטכנאי מזהה ומתעד, ולא מנסה ״לאפס״ את הכרטיס.

שאלה	תשובה
אז למה לא להשתמש ב-PIN קצר גם כסיסמה לדומיין?	כי סיסמה נבדקת מול Hash שיכול לדלוף, ואז אין מונה שעוצר ניחושים. PIN קצר בטוח רק כשהבדיקה בחומרה שסופרת ניסיונות.

שקופית 15 · כשכרטיס חכם לא עובד

על המסך: חמישה שלבי בדיקה מלמטה למעלה, ובקופסה נפרדת איך היקף התקלה מצביע על החשוד.

רקע. כמו בכל איתור תקלות בקורס: קודם החומרה, אחר כך השירות, ורק בסוף התעודה והזהות.

שלב	בודקים	איך
1	**הקורא מזוהה**	מנהל ההתקנים, תחת Smart card readers. נורית בקורא, כבל וחיבור.
2	**השירות פועל**	השירות Smart Card (SCardSvr) ב-services.msc. השירות Certificate Propagation מעתיק את התעודות מהכרטיס למאגר של המשתמש, וגם אותו כדאי לבדוק.
3	**הכרטיס נקרא**	certutil -scinfo מציג את הקורא, את הכרטיס ואת התעודות שעליו, ומבקש PIN כדי לבדוק את המפתח.
4	**התעודה תקינה**	בתוקף, שרשרת מלאה, לא בוטלה.
5	**PIN וזמן**	ה-PIN לא חסום, והשעון מסונכרן לדומיין.

ההיקף מצביע על החשוד. אם הכרטיס לא מזוהה באף תחנה, החשוד הוא הקורא או מנהל ההתקן. אם רק בתחנה אחת, השירות או התקנה מקומית. אם רק משתמש אחד נכשל, הכרטיס או התעודה שלו.

certutil -scinfo היא הבדיקה החזקה ביותר כאן, כי היא עוברת דרך כל השכבות בבת אחת. אם אין כרטיס לתרגול, הראו את הפקודה ואת השירות בלבד.

> **מה מעבירים הלאה**
>
> שחרור PIN חסום, החלפת כרטיס והנפקת תעודה נעשים בידי הגורם המוסמך ולפי נוהל היחידה. הטכנאי מעביר תסמין, היקף (כמה תחנות וכמה משתמשים), ואת הפלט של certutil -scinfo.

תרגול בצופן החלפה וכללים לשטח

שקופית 16 · אתב״ש וקיסר: קידוד או הצפנה?

על המסך: גלגל עם שתי טבעות. החיצונית היא האלפבית הגלוי והפנימית המוצפן. כפתורים לאתב״ש ולקיסר, סליידר לגודל ההזזה, ושלוש שאלות.

איך להעביר. כאן החניכים עובדים בעצמם. תנו להם כמה דקות לשחק בגלגל, בזוגות אם אפשר, ולענות על השאלות לפני שאתם מגלים תשובות. השאלות נשענות על שקופית 2 (מה ההבדל בין קידוד להצפנה) ועל שקופית 4 (השיטה ציבורית, המפתח סודי).

רקע. אתב״ש מחליף כל אות באות שבמקום המקביל מהסוף: א↔ת, ב↔ש, ג↔ר, וכן הלאה. השם עצמו בנוי מהזוגות הראשונים. בספר ירמיהו מופיע ״ששך״ במקום ״בבל״ (כה, כו, ושוב בנא, מא), ו״לב קמי״ במקום ״כשדים״ (נא, א). צופן קיסר מזיז כל אות במספר קבוע של מקומות. לפי ההיסטוריונים הרומאים, יוליוס קיסר השתמש בהזזה של שלושה. שניהם מקרים פרטיים של צופן החלפה: כל אות מוחלפת תמיד באותה אות אחרת.

תשובות

1. אתב״ש: קידוד או הצפנה? מה המפתח? באתב״ש אין מפתח. הכלל קבוע, ומי שמכיר אותו קורא כל הודעה. לפי ההגדרה משקופית 2 זה קידוד, בדיוק כמו Base64. היסטורית קראו לזה צופן, כי בזמנו השיטה עצמה הייתה הסוד. אם חניך עונה ״הצפנה, כי לא רואים מה כתוב״, זה הרגע לחזור לשקופית 2: זה שלא מבינים מה כתוב לא אומר שזו הצפנה.

2. כמה מפתחות יש לקיסר, וכמה זמן לנסות את כולם? המפתח הוא גודל ההזזה. באלפבית של 22 אותיות יש 21 הזזות שמשנות משהו (הזזה של 0 משאירה את הטקסט כמו שהוא). ביד, לנסות את כולן לוקח כמה דקות, ובמחשב פחות משנייה. לקיסר יש מפתח, אז לפי ההגדרה זו הצפנה, אבל חלשה מאוד: מרחב המפתחות קטן מדי.

3. בהחלפה כללית יש $22! \approx 1.1\times10^{21}$ מפתחות. למה גם אותה שוברים בקלות? בהחלפה כללית כל אות יכולה להפוך לכל אות, ולכן מספר המפתחות עצום, ואי אפשר לנסות את כולם. אבל כל אות הופכת תמיד לאותה אות, ולכן האות הנפוצה בעברית נשארת הנפוצה ביותר גם בטקסט המוצפן. ניתוח תדירויות (שקופית 19) מוצא את המפתח בלי לנסות את כולו, ולכן מספר המפתחות לא עוזר.

> **הקשר לעקרון קרקהוף**
>
> באתב״ש כל הסוד הוא השיטה. בהחלפה כללית השיטה ידועה, ובכל זאת היא נשברת, כי הטקסט המוצפן שומר את התדירויות. אלגוריתם מודרני צריך להחזיק גם כשהאויב יודע בדיוק איך הוא עובד וגם כשיש לו הרבה טקסט מוצפן, ואת זה בודקים לפני שמאשרים אלגוריתם חדש.

אפשר לחזור עכשיו לשקופית 1 ולשאול: מה ראינו בשער, הצפנה או קידוד?

שקופית 17 · כללים לשטח

על המסך: כללים קצרים שמסכמים את חלק א׳.

אפשר לבקש מכל משתתף לבחור כלל אחד ולתת דוגמה מהשטח. כמה הרחבות שיעזרו לכם לתקן תשובות:

- **Base64, Hex ודומיהם** הם צורת כתיבה. דוגמה טובה: סקריפט עם ״סיסמה מוסתרת״.
- **השיטה ציבורית ורק המפתח סודי.** ספק שמוכר ״אלגוריתם הצפנה ייחודי משלנו״ צריך להדליק נורה אדומה.
- **Hash מגלה שינוי**, למשל בבדיקת קובץ שהורד. הוא לא מסתיר ולא מוכיח מי שלח.

- **את הנתונים מצפינים בסימטרית**, ואסימטרית משמשת להחלפת מפתח ולחתימה. בכל חיבור HTTPS ו-VPN יש את שתיהן.
- **המנעול** אומר שהערוץ מוצפן ושהשרת הוא בעל השם שבכתובת. הוא לא אומר שאפשר לסמוך על האתר.
- **תעודה נבדקת כשרשרת**: שם, תוקף, חתימה ושורש. שעון שגוי בתחנה נראה כמו תעודה שפגה.
- **כרטיס חכם**: המפתח לא יוצא מהשבב וה-PIN נבדק בכרטיס. כרטיס חסום משחררים לפי נוהל היחידה.

חלק ב׳: פיצוח תדירויות ו-SHA-256 מבפנים

שקופית 18 · איך זה עובד מבפנים

שקופית מעבר. כאן אפשר לפצל למפגש שני. מכאן ההדגמות מחשבות את האלגוריתמים עצמם, כך שמה שרואים על המסך הוא באמת מה שקורה. המספרים ב-RSA ובדיפי-הלמן קטנים בכוונה, כדי שאפשר יהיה לבדוק אותם בראש או במחשבון. בסוף החלק יש שקופית שמפרטת מה ההדגמות מפשטות (שקופית 29).

שקופית 19 · ניתוח תדירויות שובר צופן החלפה

על המסך: טקסט באנגלית שהוצפן בקיסר, גרף של תדירות האותיות בניסיון הפענוח, וקווים מקווקווים של התדירות הרגילה באנגלית. סליידר להזזה, וכפתור לפיצוח אוטומטי.

רקע. בכל שפה יש אותיות נפוצות ואותיות נדירות. באנגלית E היא קרוב ל-13% מהאותיות, ו-Z פחות מאחוז אחד. צופן החלפה משנה את שמות האותיות אבל לא את הכמויות שלהן, ולכן צורת הגרף נשמרת והיא רק זזה. מי שמסתכל על הטקסט המוצפן ורואה שאות מסוימת מופיעה הכי הרבה, יכול לנחש שהיא E. השיטה מתוארת כבר אצל המלומד הערבי אל-כִּנדי במאה ה-9.

הטקסט כאן באנגלית כי טבלת התדירויות שלה מוכרת וזמינה. על עברית זה עובד בדיוק אותו דבר.

ההדגמה. גררו את הסליידר לאט: העמודות זזות כמקשה אחת, והצורה שלהן לא משתנה. כשהן ״נוחתות״ על הקווים המקווקווים, הטקסט נעשה קריא. ״פצח אוטומטית״ מנסה את כל 26 ההזזות. לכל הזזה הוא מחשב ציון χ^2 (חי בריבוע): לכל אות לוקחים את ההפרש בין מה שנספר למה שמצופה באנגלית, מעלים בריבוע, מחלקים במצופה ומסכמים. ציון נמוך פירושו התפלגות דומה לאנגלית. בטקסט הארוך הפיצוח נוחת על הזזה 7 והטקסט נקרא.

אחר כך לחצו ״טקסט קצר״ ושוב ״פצח״: הניחוש יוצא שגוי (הזזה 16). זה בכוונה. עם שלוש-ארבע מילים אין מספיק אותיות כדי שהסטטיסטיקה תעבוד, וזה מסביר למה הודעות קצרות ״שרדו״ יותר מארוכות.

בהחלפה כללית, שבה כל אות מוחלפת באות שרירותית, הרעיון זהה, רק מתאימים כל אות בנפרד, ואחר כך משפרים לפי צמדי אותיות ומילים נפוצות. זו התשובה לשאלה 3 בתרגול.

שאלה	תשובה
אפשר לפצח AES ככה?	לא. ב-AES כל ביט בפלט תלוי בכל ביט בקלט ובמפתח (שקופיות 20 ו-23), ולכן הטקסט המוצפן לא שומר שום צורה של השפה.

שקופית 20 · איך SHA-256 מערבב את הקלט

על המסך: רשת של 256 ריבועים, אחד לכל ביט בפלט. ריבועים שהתהפכו בהקשה האחרונה מסומנים בצהוב. בצד תרשים של מבנה האלגוריתם, ובתחתית מונה ביטים וגודל הקלט.

רקע. SHA-256 עובד בבלוקים של 512 ביט (64 בתים). קודם מרפדים את ההודעה: מוסיפים ביט 1, אחריו אפסים, ובסוף את אורך ההודעה המקורית ב-64 ביט, כך שהאורך הכולל יתחלק ב-512. אחר כך כל בלוק עובר ״פונקציית דחיסה״ של 64 סבבים, שמערבבת אותו עם מצב פנימי של 256 ביט. המצב עובר מבלוק לבלוק, והמצב אחרי הבלוק האחרון הוא ה-Hash.

בגלל הריפוד, הודעה של עד 55 בתים נכנסת בבלוק אחד, ומ-56 בתים צריך שני בלוקים. הקלידו ותראו את זה בשורה התחתונה. אות עברית היא שני בתים ב-UTF-8, ולכן מילה עברית מגיעה לגבול מהר יותר.

שלוש הדרישות שעל המסך הן מה שמגדיר Hash טוב: אי אפשר לחזור מהפלט לקלט; לא מוצאים שני קלטים עם אותו פלט; וכל ביט בפלט תלוי בכל ביט בקלט.

ההדגמה. הקלידו מילה אות אחר אות. בכל הקשה מתהפכים בערך חצי מהריבועים, גם כשמוסיפים רק רווח. הדפוס לא אקראי, כי אותה מילה תמיד נותנת אותו דפוס, אבל אי אפשר לצפות אותו מראש.

שאלה	תשובה
מה ההבדל בין SHA-1 ל-SHA-256?	SHA-1 מחזיר 160 ביט ונמצאו בו התנגשויות. SHA-256 מחזיר 256 ביט, ולא ידועה בו שום חולשה מעשית.

XOR ומפתח זרם

שקופית 21 · אותה פעולת XOR מצפינה ומפענחת

על המסך: טקסט של עד שלושה תווים, מפתח אקראי, וחמש שורות ביטים שמתמלאות עמודה אחרי עמודה: טקסט, מפתח, מוצפן, שוב מפתח, וטקסט.

רקע. XOR (״או בלעדי״, מסומן ⊕) היא פעולה על שני ביטים: התוצאה 1 אם הביטים שונים, ו-0 אם הם שווים. דרך פשוטה להסביר: ביט מפתח 1 הופך את ביט הטקסט, וביט מפתח 0 משאיר אותו. מכאן שהפעלה פעמיים של אותו מפתח מחזירה את המקור: $(P \oplus K) \oplus K = P$. לכן אותה פעולה בדיוק מצפינה ומפענחת.

פנקס חד-פעמי. אם המפתח אקראי באמת, ארוך כמו ההודעה ומשמש פעם אחת בלבד, ההצפנה בלתי ניתנת לשבירה. קלוד שאנון הוכיח את זה ב-1949: כל טקסט באותו אורך אפשרי באותה מידה, ולכן הטקסט המוצפן לא מגלה כלום. הבעיה מעשית: צריך להעביר מראש מפתח אקראי באורך של כל מה שתשלחו אי פעם.

שימוש חוזר במפתח. אם מצפינים שתי הודעות באותו מפתח, מספיק לעשות XOR בין שני הטקסטים המוצפנים: $C_1 \oplus C_2 = P_1 \oplus P_2$. המפתח נעלם מהחשבון, ונשאר שילוב של שתי ההודעות, שממנו אפשר לחלץ את שתיהן בעזרת ניחושים על התוכן. זה קרה בפועל: בפרויקט ״ונונה״ האמריקאים פענחו מברקים סובייטיים כי חלק מהפנקסים החד-פעמיים יוצרו פעמיים.

מכאן השאלה שהשקופית הבאה עונה עליה: אם אי אפשר להחזיק מפתח באורך כל התעבורה, ואסור לחזור על מפתח, איך מצפינים גיגה-בייטים?

שקופית 22 · מפתח זרם

על המסך: מפתח של 16 בתים, מספר חד-פעמי של 12 בתים והודעה. טבלה שמראה איך כל ערך של מונה הופך ל-16 בתים של ״זרם״, והטקסט המוצפן. בפאנל השני: הודעה שנייה באותו מפתח, עם אותו מספר חד-פעמי או עם מספר אחר.

רקע. הרעיון: במקום להחזיק מפתח באורך ההודעה, מייצרים אותו. לוקחים מפתח קצר (כאן 128 ביט) ופונקציה שמערבבת היטב, כמו AES. לכל בלוק של 16 בתים בונים קלט שמורכב מ״מספר חד-פעמי״ (12 בתים, nonce) ומונה (4 בתים), ומצפינים אותו ב-AES עם המפתח. הפלט הוא 16 בתים שנראים אקראיים לגמרי. המונה עולה באחד לבלוק הבא, וכך הלאה, עד שיש רצף באורך ההודעה. את הרצף הזה, ״מפתח הזרם״, מצרפים להודעה ב-XOR, בדיוק כמו בפנקס חד-פעמי. השיטה נקראת מצב מונה (CTR), והיא הבסיס של AES-GCM, שבו משתמשים ב-TLS, ב-VPN ובהרבה מערכות אחרות.

למה הזרם לא חוזר. AES עם מפתח קבוע נותן פלט שונה לכל קלט שונה. בתוך הודעה המונה תמיד עולה, ובין הודעות המספר החד-פעמי שונה, ולכן אף קלט ל-AES לא חוזר, וגם אף בלוק בזרם. מונה של 32 ביט מספיק ל-2^{32} בלוקים של 16 בתים, כ-64 ג׳יגה-בייט בהודעה אחת. TLS מחלק את התעבורה לרשומות של עד 16 קילובייט, ולכל רשומה יש מספר חד-פעמי משלה. ב-TLS 1.3 המספר החד-פעמי נגזר ממספר הרשומה, כך שהוא לא יכול לחזור, ואחרי כמות גדולה מאוד של רשומות הצדדים מחליפים מפתח. כך מצפינים גיגה-בייטים של תעבורה בלי שהזרם יחזור פעם אחת.

ChaCha20, שמשמש בעיקר בטלפונים ובמעבדים בלי האצת AES בחומרה, בונה זרם באותו עיקרון, עם פונקציית ערבול אחרת.

ההדגמה. החישוב אמיתי: AES-128 במצב מונה. הקלידו בשדה ההודעה: כל 16 בתים מתווסף בלוק, והמונה (בצהוב בטבלה) עולה באחד. ״🎲״ ליד המפתח או ליד המספר החד-פעמי מחליף אותם, וכל הזרם משתנה. AES עצמו מוסבר בשקופית הבאה. כאן מספיק להתייחס אליו כמערבל עם מפתח: קלט ששונה בביט אחד נותן פלט שונה לגמרי.

עכשיו הפאנל השני. ״אותו מספר חד-פעמי״ מצפין את ההודעה השנייה באותו זרם. השורה $C_1 \oplus C_2 \oplus P_1$ מראה מה יקבל תוקף שיש לו שני טקסטים מוצפנים ויודע (או מנחש) את הראשון: את ההודעה השנייה, בלי המפתח. זו בדיוק הסכנה מהקופסה האדומה בשקופית 21. ״מספר חד-פעמי אחר״ מחליף את הזרם של ההודעה השנייה, ואותו חישוב נותן רעש.

בשטח. ב-2017 פורסמה מתקפה על WPA2 בשם KRACK, שגרמה למכשירים להשתמש שוב במספר חד-פעמי, ואיתו באותו זרם. התיקון היה עדכון תוכנה בלקוחות ובנקודות הגישה. זו דוגמה טובה לכך שגם אלגוריתם חזק נופל כשמספר חד-פעמי חוזר.

שאלה	תשובה
אם הזרם נגזר מהמפתח, הוא לא באמת אקראי. אז למה זה בטוח?	הוא לא אקראי, ולכן זה לא פנקס חד-פעמי עם הוכחה מתמטית. הביטחון נשען על כך שאף אחד לא יודע להבדיל בין פלט של AES לרצף אקראי בלי לדעת את המפתח, ואת המפתח (2^{128} אפשרויות) אי אפשר לנחש.
המספר החד-פעמי צריך להיות סודי?	לא. הוא נשלח גלוי עם ההודעה, או נגזר ממספר הרשומה. הדרישה היחידה היא שלא יחזור עם אותו מפתח.
זה מה שנקרא ״ניפוח מפתח״?	לא בדיוק. מה שמוצג כאן הוא מפתח זרם. ״מתיחת מפתח״ (key stretching) היא הפיכת סיסמה למפתח בפונקציה איטית בכוונה (שקופית 5). ב-TLS יש גם ״גזירת מפתחות״: מהסוד המשותף מחשבים כמה מפתחות נפרדים, לכל כיוון ולכל שלב.

טעות נפוצה

לחשוב שמפתח זרם מגן גם משינוי. הוא לא. ביט שמתהפך בטקסט המוצפן מתהפך באותו מקום בטקסט אחרי הפענוח, ואף אחד לא יודע. לכן GCM ו-ChaCha20-Poly1305 מוסיפים לכל הודעה תג שמגלה כל שינוי. ההדגמה מוותרת על התג, וזה כתוב בשקופית 29.

AES, RSA, דיפי-הלמן וחתימה במספרים

שקופית 23 · מה קורה בתוך AES

על המסך: בלוק של 16 בתים בטבלה של 4×4, שמוצפן צעד אחרי צעד. כפתורים לנגינה ולצעידה, הסבר לכל צעד, וכפתור שהופך ביט אחד במפתח.

רקע. AES נבחר כתקן ההצפנה הסימטרית של ארה״ב ב-2001, אחרי תחרות פתוחה, ומאז הוא ההצפנה הסימטרית הנפוצה בעולם. הוא מצפין בלוקים של 128 ביט (16 בתים), שמסודרים בטבלה של ארבע שורות על ארבע עמודות. המפתח יכול להיות 128, 192 או 256 ביט, ולפי זה יש 10, 12 או 14 סבבים. בהדגמה AES-128, עשרה סבבים.

לפני הסבב הראשון מצרפים לבלוק את המפתח ב-XOR. כל סבב מורכב מארבעה צעדים:

- **SubBytes** מחליף כל בית לפי טבלה קבועה. זה החלק הלא-ליניארי, שבלעדיו אפשר היה לפתור את הצופן במשוואות.
- **ShiftRows** מזיז כל שורה מספר מקומות שונה. השורה העליונה לא זזה.
- **MixColumns** מערבב את ארבעת הבתים בכל עמודה, כך שכל בית משפיע על כל העמודה.
- **AddRoundKey** מצרף ב-XOR את מפתח הסבב. מפתחות הסבבים נגזרים מהמפתח המקורי.

בסבב האחרון אין MixColumns. אחרי שניים-שלושה סבבים כל בית בפלט כבר תלוי בכל בית בקלט.

ההדגמה. נגנו את כל עשרת הסבבים פעם אחת, ואחר כך חזרו להתחלה וצעדו ידנית בסבב הראשון. שימו לב איך SubBytes משנה את כל 16 התאים, ו-ShiftRows רק מזיז אותם. ״הפוך ביט אחד במפתח״ מצפין את אותו בלוק במפתח ששונה בביט אחד, ומראה שכמחצית מ-128 ביטי הפלט משתנים.

שאלה	תשובה
אפשר לפצח AES-128 בכוח, לנסות את כל המפתחות?	יש 2^{128} מפתחות, מספר בן 39 ספרות. אין שום דרך מעשית לנסות את כולם, גם לא בכל המחשבים בעולם יחד.
אז למה יש גם AES-256?	מרווח ביטחון, בעיקר מול מחשב קוונטי עתידי (שקופית 29). ל-AES-128 לא ידועה חולשה מעשית.

שקופית 24 · RSA במספרים קטנים

על המסך: שדות ל-p, q ו-e, החישוב של n, φ ו-d, הצפנה ופענוח של מספר, וכפתור ״פרק את n כמו תוקף״.

רקע. בוחרים שני מספרים ראשוניים, p ו-q, ומכפילים: $n = p \cdot q$. מחשבים $\varphi = (p-1)(q-1)$, בוחרים מעריך ציבורי e שאין לו גורם משותף עם φ, ומחשבים את d, המספר שמקיים $e \cdot d \bmod \varphi = 1$. המפתח הציבורי הוא (n, e) והפרטי הוא d. הצפנה: $c = m^e \bmod n$. פענוח: $m = c^d \bmod n$.

כל הביטחון נשען על כך שקשה לפרק את n לגורמים. מי שיודע את p ואת q מחשב את φ ומשם את d. ב-RSA אמיתי n בן 2048 ביט, כ-617 ספרות עשרוניות. המספר הגדול ביותר מהסוג הזה שפורק בפומבי הוא בן 829 ביט, ב-2020, בחישוב של אלפי שנות מעבד.

ההדגמה. ברירת המחדל היא הדוגמה הקלאסית: p=61, q=53, e=17. מכאן n=3233, φ=3120 ו-d=2753 (בדיקה: 17×2753 = 46801 = 15×3120 + 1). ההודעה 65 מוצפנת ל-2790. שנו את p ו-q וראו ש-d משתנה לגמרי. ״פרק את n כמו תוקף״ מוצא את p ו-q בשבריר שנייה, ומשם את d. נסו גם e=3 עם p=61 ו-q=31: φ=1800 מתחלק ב-3, ולכן אין d, וההדגמה מסמנת את זה באדום.

שאלה	תשובה
איזה e משתמשים בו בפועל?	כמעט תמיד 65537. הוא ראשוני, ובזכות המבנה הבינארי שלו (שני ביטים דולקים) החישוב איתו מהיר.
למה לא משתמשים רק ב-RSA?	הוא איטי ומוגבל בגודל ההודעה. משתמשים בו לחתימה, ובמערכות ישנות יותר גם להעברת מפתח.

שקופית 25 · איך שני צדדים מסכימים על סוד בערוץ פתוח

על המסך: אנימציה של ערבוב צבעים, ומתחתיה אותו דבר במספרים, עם סליידרים לסודות של שני הצדדים.

רקע. דיפי-הלמן (1976) פותר בעיה שנראית בלתי אפשרית: שני צדדים שמעולם לא נפגשו מגיעים לאותו סוד, כשכל מה שהם שולחים גלוי למאזין. במספרים: מסכימים בגלוי על ראשוני p ועל בסיס g. דנה בוחרת סוד a ושולחת $A = g^a \bmod p$. יוסי בוחר סוד b ושולח $B = g^b \bmod p$. דנה מחשבת $B^a \bmod p$, יוסי מחשב $A^b \bmod p$, ושניהם מקבלים $g^{ab} \bmod p$. המאזין רואה את p, g, A ו-B, אבל כדי לחשב את הסוד הוא צריך את a או את b, כלומר לפתור את בעיית ״הלוגריתם הבדיד״. במספרים גדולים אין לה פתרון ידוע בזמן סביר.

סיפור הצבעים הוא אותו רעיון: קל לערבב צבעים, וכמעט אי אפשר להפריד תערובת לצבעים שלה.

ההדגמה. ברירת המחדל: $p=23$, $g=5$, $a=6$, $b=15$, ולכן $A=8$, $B=19$ והסוד המשותף 2. הזיזו את a ו-b ובדקו שהסוד תמיד זהה בשני הצדדים.

TLS 1.3-ב משתמשים בגרסה על עקומים אליפטיים (ECDHE), ומייצרים סודות חדשים בכל חיבור ומוחקים אותם בסופו. בגלל זה גם מי שיגנוב בעתיד את המפתח הפרטי של התעודה לא יוכל לפענח הקלטות ישנות. זה נקרא ״סודיות קדימה״.

> **מה דיפי-הלמן לא נותן**
>
> הוא לא יודע מי בצד השני. תוקף באמצע יכול לעשות דיפי-הלמן נפרד עם כל צד ולהעביר ביניהם. לכן ב-TLS השרת חותם על ההחלפה במפתח של התעודה שלו (CertificateVerify, שקופית 27).

שקופית 26 · חתימה דיגיטלית במספרים

על המסך: מסמך, וכל שלב בחתימה ובבדיקה מוצג במספרים, עם אותו מפתח RSA משקופית 24.

רקע. זו ההדגמה משקופית 10, עם החשבון גלוי. h הוא ה-Hash של המסמך (SHA-256 אמיתי) אחרי צמצום מודולו 3233, כדי שיהיה קטן מ-n. החתימה היא $s = h^d \bmod n$, ורק מי שמחזיק את d יכול לחשב אותה. הבדיקה: $s^e \bmod n$, ומשווים ל-h. כל אחד יכול לבדוק, כי e ו-n ציבוריים.

שנו אות במסמך: ה-Hash משתנה, ולכן גם h, והבדיקה נכשלת. בגלל הצמצום ל-3233 יש בערך סיכוי אחד ל-3233 ששינוי מקרי ״יעבור״. זו מגבלה של ההדגמה בלבד. ב-RSA אמיתי לא מצמצמים את ה-Hash, אלא מרפדים אותו בשיטה מוגדרת (PKCS#1 v1.5 ברוב התעודות, PSS בלחיצת היד של TLS 1.3) וחותמים על התוצאה.

לחיצת היד של TLS 1.3, מגבלות וסיכום

שקופית 27 · לחיצת היד של TLS 1.3

על המסך: תרשים של הודעות בין לקוח לשרת, צעד אחרי צעד. הצבע של כל הודעה אומר במה היא מוגנת, ובצד מוסבר מה המאזין רואה בכל צעד.

רקע. לחיצת היד מחברת את כל מה שנלמד עד עכשיו: דיפי-הלמן להסכמה על סוד, תעודה וחתימה לזהות השרת, Hash לבדיקה ששני הצדדים ראו אותן הודעות, ובסוף הצפנה סימטרית. מהסוד המשותף גוזרים קודם מפתחות ללחיצת היד עצמה, ואחר כך מפתחות נפרדים למידע.

צעד	הודעה	מה חשוב להגיד
1-2	**ClientHello, ServerHello**	הלקוח שולח גרסאות, צפנים, שם האתר ואת החצי הציבורי שלו של דיפי-הלמן (key_share). השרת בוחר צופן ושולח את החצי שלו. הכול גלוי, אבל החצאים הציבוריים לא מגלים את הסוד.
3	**חישוב מקומי**	כל צד מגיע לאותו סוד, ממנו נגזרים מפתחות לחיצת היד. שום מפתח לא עובר ברשת.
4-5	**EncryptedExtensions, Certificate**	מכאן כל מה שהשרת שולח מוצפן, כולל התעודה. ב-TLS 1.2 התעודה עברה גלויה.
6	**CertificateVerify**	השרת חותם במפתח הפרטי של התעודה על כל ההודעות עד כה, כולל ה-key_share. כך הוא מוכיח שהוא מחזיק את המפתח, ותוקף שהחליף את ה-key_share בדרך נכשל כאן.
7-8	**Finished בשני הכיוונים**	קוד אימות (HMAC) על כל לחיצת היד. לפני שהלקוח שולח את שלו, הוא בודק את שרשרת התעודה כמו בחלק א׳. אם משהו נכשל, זה הרגע שבו מופיעה שגיאת התעודה בדפדפן.
9-10	**מידע**	מוצפן במפתחות היישום, במפתח זרם עם מספר חד-פעמי לכל רשומה (שקופית 22). המידע הראשון יוצא אחרי סבב הלוך-חזור אחד (1-RTT).

יש גם מצב 0-RTT לחיבור חוזר לשרת מוכר, שבו הלקוח שולח מידע כבר בהודעה הראשונה. הוא לא מוצג בהדגמה.

בשיעור 3 ברשתות לחיצת היד הוצגה בקצרה כ״Hello, תעודה, גזירת מפתחות ו-Finished״. כאן רואים למה כל חלק נחוץ.

שקופית 28 · מה המאזין רואה ב-TLS 1.2 וב-TLS 1.3

על המסך: טבלה עם מתג בין שתי הגרסאות. שורות שהשתנו מסומנות.

מה השתנה ב-1.3: התעודה של השרת מוצפנת. החלפת מפתחות ב-RSA ישיר, שבה הלקוח מצפין את הסוד במפתח של השרת, בוטלה, ונשאר רק דיפי-הלמן זמני. ב-RSA ישיר, מי שמקליט תעבורה ומשיג אחר כך את מפתח השרת מפענח את כל ההקלטות, ובדיפי-הלמן זמני לא. ולחיצת היד קצרה בסבב הלוך-חזור אחד.

מה לא השתנה: כתובות IP ופורטים, הגרסה ורשימת הצפנים, שם האתר (SNI), וגודל ותזמון התעבורה. גם מאלה לבד אפשר לפעמים להסיק לא מעט. הרחבה חדשה, ECH, מצפינה גם את שם האתר, אבל היא עוד לא נתמכת בכל מקום.

בשטח. בגלל השינויים האלה, לוכד תעבורה (כמו Wireshark) מראה ב-1.3 פחות מבעבר. אי אפשר לראות בו את התעודה שהשרת שלח, ולכן בתקלות תעודה בודקים בצד הלקוח, בדפדפן או ב-certutil.

שקופית 29 · מגבלות ההדגמות

לא לדלג על השקופית הזו. חניכים יוצאים לפעמים מהשיעור עם הרושם ש"RSA זה החשבון הזה". החשבון נכון, אבל בלי ריפוד, בלי גדלים אמיתיים ובלי מצבי הפעלה הוא לא בטוח.

- **RSA**: מספרים זעירים ובלי ריפוד. גם במספרים גדולים, RSA בלי ריפוד שביר: אותה הודעה תמיד נותנת אותו טקסט מוצפן, ויש מתקפות שמנצלות את המבנה המתמטי. בפועל: 2048 ביט ומעלה, OAEP להצפנה, ו-PKCS#1 v1.5 או PSS לחתימה.
- **AES ומפתח זרם**: שקופית 23 מצפינה בלוק בודד, ושקופית 22 מייצרת זרם בלי בדיקת שלמות. בפועל משתמשים ב-GCM או ב-ChaCha20-Poly1305, שמוסיפים לכל רשומה תג שמגלה כל שינוי.
- **כרטיס חכם ו-TLS**: התרשימים מראים את העיקרון. הפרוטוקולים האמיתיים כוללים עוד שדות, בדיקות ומקרי קצה.

מחשב קוונטי. מחשב קוונטי גדול מספיק, אם וכאשר ייבנה, ישבור את RSA ואת ECC, כי הוא יודע לפרק לגורמים ולפתור לוגריתם בדיד ביעילות. על AES ועל SHA-256 ההשפעה קטנה בהרבה, ובערך מחצית מאורך המפתח האפקטיבי, ולכן AES-256 נחשב בטוח גם אז. המעבר לאלגוריתמים עמידים כבר התחיל: דפדפנים נפוצים כבר משלבים בלחיצת היד החלפת מפתחות היברידית עם ML-KEM, אלגוריתם שתוקנן ב-2024. כי השיטות ציבוריות, אפשר להחליף אלגוריתם בלי לבנות את המערכת מחדש.

מה כן אמין: SHA-256 ו-AES בהדגמות הם האלגוריתמים המלאים, והעקרונות (מפולת, פירוק לגורמים, לוגריתם בדיד, שרשרת חתימות) הם אלה שעליהם נשענות המערכות האמיתיות.

שקופית 30 · הכלים משלימים זה את זה

על המסך: הכלים מהשיעור, ומתחתם איך הם פועלים יחד בלחיצת היד של TLS 1.3.

לסגירה, עברו על הכלים ושאלו על כל אחד מה הוא מוכיח ומה לא. אחר כך בקשו מחניך לתאר במשפט אחד מה קורה כשנכנסים לאתר בדפדפן. תשובה טובה, בערך: "הדפדפן והשרת מסכימים על מפתח בדיפי-הלמן, השרת מוכיח שהוא בעל השם בתעודה בכך שהוא חותם במפתח הפרטי שלה, הדפדפן בודק את שרשרת התעודה, ומשם כל המידע מוצפן בהצפנה סימטרית". אם חסר אחד מהחלקים, שאלו מה היה קורה בלעדיו.

מפת המצגת, ערכי בדיקה ומילון

מפת המצגת

מס׳	כותרת השקופית	מס׳	כותרת השקופית
1	הצפנה וזהות (שער)	16	אתב״ש וקיסר: קידוד או הצפנה?
2	קידוד, הצפנה והסתרה	17	כללים לשטח
3	סטגנוגרפיה מסתירה מסר בתוך תמונה	18	איך זה עובד מבפנים (מעבר)
4	Hash, סימטרית, אסימטרית וחתימה	19	ניתוח תדירויות שובר צופן החלפה
5	שינוי של תו אחד משנה את כל ה-Hash	20	איך SHA-256 מערבב את הקלט
6	הצפנה סימטרית ובעיית הפצת המפתח	21	אותה פעולת XOR מצפינה ומפענחת
7	בהצפנה אסימטרית המפתח הפרטי לא יוצא מהבעלים	22	מפתח זרם: ממפתח קצר להצפנה של גיגה-בייטים
8	בפועל משלבים את שתי השיטות	23	מה קורה בתוך AES
9	מה המנעול בדפדפן מבטיח	24	RSA במספרים קטנים
10	חתימה דיגיטלית: מוכיחה מי חתם, ושאף תו לא השתנה	25	איך שני צדדים מסכימים על סוד בערוץ פתוח
11	כל תעודה בשרשרת חתומה בידי זו שמעליה	26	חתימה דיגיטלית במספרים
12	שגיאות תעודה נפוצות ואיפה בודקים	27	לחיצת היד של TLS 1.3 בסבב הודעות אחד
13	כרטיס חכם: המפתח הפרטי נוצר בשבב ולא יוצא ממנו	28	מה המאזין רואה ב-TLS 1.2 וב-TLS 1.3
14	למה PIN של 4 ספרות מספיק בכרטיס חכם	29	מגבלות ההדגמות
15	כשכרטיס חכם לא עובד: בודקים מלמטה למעלה	30	הכלים משלימים זה את זה

ערכי בדיקה להדגמות

אם רוצים לוודא לפני השיעור שההדגמות עובדות, אלה הערכים שצריכים לצאת:

הדגמה	קלט	תוצאה
אתב״ש / קיסר 3 (שקופית 16)	בבל	ששך / ההס
PIN (שקופית 14)	שלושה ניסיונות שגויים	הכרטיס ננעל, גם 2580 לא פותח
פיצוח תדירויות (שקופית 19)	טקסט ארוך / קצר	הזזה 7 / ניחוש שגוי (16)
SHA-256 (שקופית 20)	56 בתים	שני בלוקים

RSA (שקופית 24)	p=61, q=53, e=17, m=65	n=3233, d=2753, c=2790
דיפי-הלמן (שקופית 25)	p=23, g=5, a=6, b=15	A=8, B=19, סוד 2

מילון קצר

מונח	הסבר	מונח	הסבר
Hash	טביעת אצבע באורך קבוע, חד-כיוונית.	**מלח**	ערך אקראי לכל משתמש שמצרפים לסיסמה לפני ה-Hash.
HMAC	Hash עם מפתח סודי. מוכיח שלמות ומקור.	**מתיחת מפתח**	הפיכת סיסמה למפתח בפונקציה איטית בכוונה.
מפתח זרם	רצף ארוך שנגזר ממפתח קצר ומספר חד-פעמי, ומצורף ב-XOR.	**מספר חד-פעמי**	ערך שאסור שיחזור עם אותו מפתח. לא סודי.
GCM	מצב מונה של AES עם תג שלמות.	**תעודה**	מסמך חתום שקושר מפתח ציבורי לשם.
SAN	רשימת השמות שהתעודה תקפה להם.	**CRL / OCSP**	שתי דרכים לבדוק אם תעודה בוטלה.
ECDHE	דיפי-הלמן זמני על עקומים אליפטיים.	**סודיות קדימה**	דליפת מפתח בעתיד לא חושפת הקלטות ישנות.
SNI	שם האתר בתחילת חיבור TLS. בדרך כלל גלוי.	**PKINIT**	כניסה ל-Kerberos בתעודה, למשל בכרטיס חכם.
RTT	זמן הלוך-חזור ברשת.	**עקרון קרקהוף**	השיטה ציבורית, רק המפתח סודי.